

SE3082

AWS EC2 Lab - Screenshots and Calculations

Lab 06

Exercise 4: Pragma omp (openmp) parallel for vs parallel for with collapse for nested loops

IT24610328

On instance with 2 vCPUs (t3.micro)

Nano - creating the pragma omp parallel for (plain) for a nested loop

```
GNU nano 6.2                                ex4_nested.c
#include <omp.h>
#include <stdio.h>

int main() {

#pragma omp parallel for
  for (int r=1; r<=5; r++)
    for (int k=1; k<=10; k++) {
        printf("r = %d, k = %d, thread # = %d\n", r, k, omp_get_thread_num());
    }

    return 0;
}
```

Nano - creating the pragma omp parallel for (collapse with level 2) for a nested loop

```
GNU nano 6.2                                ex4_collapse.c
#include <omp.h>
#include <stdio.h>

int main() {

#pragma omp parallel for collapse(2)
  for (int r=1; r<=5; r++)
    for (int k=1; k<=10; k++) {
        printf("r = %d, k = %d, thread # = %d\n", r, k, omp_get_thread_num());
    }

    return 0;
}
```

Outputs from running the parallel for (plain) for a nested loop code.

Run one

```
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
r = 2, k = 9, thread # = 0
r = 2, k = 10, thread # = 0
r = 3, k = 1, thread # = 0
r = 3, k = 2, thread # = 0
r = 3, k = 3, thread # = 0
r = 3, k = 4, thread # = 0
r = 3, k = 5, thread # = 0
r = 3, k = 6, thread # = 0
r = 3, k = 7, thread # = 0
r = 3, k = 8, thread # = 0
r = 3, k = 9, thread # = 0
r = 3, k = 10, thread # = 0
r = 4, k = 1, thread # = 1
r = 4, k = 2, thread # = 1
r = 4, k = 3, thread # = 1
r = 4, k = 4, thread # = 1
r = 4, k = 5, thread # = 1
r = 4, k = 6, thread # = 1
r = 4, k = 7, thread # = 1
r = 4, k = 8, thread # = 1
r = 4, k = 9, thread # = 1
r = 4, k = 10, thread # = 1
r = 5, k = 1, thread # = 1
r = 5, k = 2, thread # = 1
r = 5, k = 3, thread # = 1
r = 5, k = 4, thread # = 1
r = 5, k = 5, thread # = 1
r = 5, k = 6, thread # = 1
r = 5, k = 7, thread # = 1
r = 5, k = 8, thread # = 1
r = 5, k = 9, thread # = 1
r = 5, k = 10, thread # = 1
```

Run two

```
r = 4, k = 1, thread # = 1
r = 4, k = 2, thread # = 1
r = 4, k = 3, thread # = 1
r = 4, k = 4, thread # = 1
r = 4, k = 5, thread # = 1
r = 4, k = 6, thread # = 1
r = 4, k = 7, thread # = 1
r = 4, k = 8, thread # = 1
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
r = 2, k = 9, thread # = 0
r = 2, k = 10, thread # = 0
```

```
r = 3, k = 1, thread # = 0
r = 3, k = 2, thread # = 0
r = 3, k = 3, thread # = 0
r = 3, k = 4, thread # = 0
r = 3, k = 5, thread # = 0
r = 3, k = 6, thread # = 0
r = 3, k = 7, thread # = 0
r = 3, k = 8, thread # = 0
r = 3, k = 9, thread # = 0
r = 3, k = 10, thread # = 0
r = 4, k = 9, thread # = 1
r = 4, k = 10, thread # = 1
r = 5, k = 1, thread # = 1
r = 5, k = 2, thread # = 1
r = 5, k = 3, thread # = 1
r = 5, k = 4, thread # = 1
r = 5, k = 5, thread # = 1
r = 5, k = 6, thread # = 1
r = 5, k = 7, thread # = 1
r = 5, k = 8, thread # = 1
r = 5, k = 9, thread # = 1
r = 5, k = 10, thread # = 1
```

Outputs from running the parallel for (collapse with level 2) for a nested loop code.

Run one

```
r = 3, k = 6, thread # = 1
r = 3, k = 7, thread # = 1
r = 3, k = 8, thread # = 1
r = 3, k = 9, thread # = 1
r = 3, k = 10, thread # = 1
r = 4, k = 1, thread # = 1
r = 4, k = 2, thread # = 1
r = 4, k = 3, thread # = 1
r = 4, k = 4, thread # = 1
r = 4, k = 5, thread # = 1
r = 4, k = 6, thread # = 1
r = 4, k = 7, thread # = 1
r = 4, k = 8, thread # = 1
r = 4, k = 9, thread # = 1
r = 4, k = 10, thread # = 1
r = 5, k = 1, thread # = 1
r = 5, k = 2, thread # = 1
r = 5, k = 3, thread # = 1
r = 5, k = 4, thread # = 1
r = 5, k = 5, thread # = 1
r = 5, k = 6, thread # = 1
r = 5, k = 7, thread # = 1
r = 5, k = 8, thread # = 1
r = 5, k = 9, thread # = 1
r = 5, k = 10, thread # = 1
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
r = 2, k = 9, thread # = 0
r = 2, k = 10, thread # = 0
r = 3, k = 1, thread # = 0
r = 3, k = 2, thread # = 0
r = 3, k = 3, thread # = 0
r = 3, k = 4, thread # = 0
r = 3, k = 5, thread # = 0
```

Run two

```
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
r = 2, k = 9, thread # = 0
r = 2, k = 10, thread # = 0
r = 3, k = 1, thread # = 0
r = 3, k = 2, thread # = 0
r = 3, k = 3, thread # = 0
r = 3, k = 4, thread # = 0
r = 3, k = 5, thread # = 0
r = 3, k = 6, thread # = 1
r = 3, k = 7, thread # = 1
r = 3, k = 8, thread # = 1
r = 3, k = 9, thread # = 1
r = 3, k = 10, thread # = 1
```

```
r = 4, k = 1, thread # = 1
r = 4, k = 2, thread # = 1
r = 4, k = 3, thread # = 1
r = 4, k = 4, thread # = 1
r = 4, k = 5, thread # = 1
r = 4, k = 6, thread # = 1
r = 4, k = 7, thread # = 1
r = 4, k = 8, thread # = 1
r = 4, k = 9, thread # = 1
r = 4, k = 10, thread # = 1
r = 5, k = 1, thread # = 1
r = 5, k = 2, thread # = 1
r = 5, k = 3, thread # = 1
r = 5, k = 4, thread # = 1
r = 5, k = 5, thread # = 1
r = 5, k = 6, thread # = 1
r = 5, k = 7, thread # = 1
r = 5, k = 8, thread # = 1
r = 5, k = 9, thread # = 1
r = 5, k = 10, thread # = 1
```

How the `pragma omp parallel for (plain)` for a nested loop works in the program.

With plain `#pragma omp parallel for`, only the outer loop (r, with 5 iterations) is divided among the available threads. Each thread that gets assigned an r value then runs the entire inner loop (k, 10 iterations) sequentially by itself. This means with only 2 threads on this instance, work is split at most 5 ways across the outer loop, but since there are more outer iterations than threads, each thread ends up handling multiple full r blocks one after another.

How the `pragma omp parallel for (collapsed with level 2)` for a nested loop works in the program.

With `#pragma omp parallel for collapse(2)`, both loops are merged into a single combined iteration space of $5 \times 10 = 50$ total iterations before OpenMP distributes any work. This means threads are no longer limited by the outer loop's iteration count (5), work can be split much more evenly across all 50 individual (r,k) combinations, which is especially useful when the outer loop has too few iterations to meaningfully use all available threads on its own.

The main difference when you switch to the collapsed version from the plain for

The plain version consistently splits at a whole-r boundary (r=3 to r=4), while the collapsed version splits mid-way through r=3 (k=5 to k=6). It directly showing that collapse treats the nested loops as one flattened space rather than dividing only the outer loop